

ARD-legacy: 行锚图

定义与证明资料 / 中文版

这是保留的旧版规则；下文历史名称 ARD 均指 ARD-legacy。默认新版见 ARD。

发布定义，2026-09-13。PDF · NER 弧线图展开器 · Python · 良序证明 · 普通 Lean 入口。

ARD 即 **Anchored Row Diagrams (行锚图)**。一个式子是有限图，不是操作路径。它的行标是更早列的地址；复制来源块时，行锚与根、父、子列一起移动。软件中的资源保护不是数学规则。

1. 有限语法与根闭包

有限图是列串 $G = (C_0, \dots, C_{m-1})$ 。第 j 列中的压缩组为

$$(k, p, q), \quad 0 \leq k < j, \quad 0 \leq q \leq p < j.$$

三个坐标依次是**行锚**、**父列**、**最大根**，子列由该组所在列确定。不要求 $k \leq p$ ：行锚可以在父列之后，但必须在子列之前。

此组表示所有 $0 \leq t \leq q$ 的原子关系 (k, t, p, j) 。这是根向下闭包，不是行锚向下闭包。同一 (k, p) 只保留最大的 q 。忽略重复关系，但保留空列。零图没有列，写作 $\emptyset$ ； $[]$ 是一个空列，表示自然数一。自然数 m 由 m 个空列组成。

无损列表视图中，每对方括号包住一列。例如

```
[[[(0,0,0)][(1,1,1)]]
```

是三列种子 A_3 。Python 与 NER 保存最大根，Lean 有限几何保存完整根闭包；两者编码相同的有限关系。

2. 比较

每列中的压缩组按键 (p, k, q) 递减排序，然后按该键对列表作字典序比较。图从最左列开始逐列作字典序比较。两个层级都规定真前缀较小。因此只需找到最早不同列及其中最早不同组，不计算展开、可达性、操作路径或全图支持信息。

若将所有根显式列出并按 (p, k, t) 排序，所得比较相同：对固定 (p, k) ，最先不同的最大根已在到达更小组之前决定结果。

另加最大外顶端 Top，显示为 Limit of ARD。它不是有限列，也不是行锚。

3. 基本列

指标 n 为非负整数。定义 $0[n] = 0$ ；此恒等式不算严格下降步骤。对非空图置 $x = m - 1$ ，以 ∂G 表示删除末列，并规定

$$G[0] = \partial G, \quad G[n] = \partial G \quad \text{若 } C_x = \emptyset, \text{ 则对所有 } n \text{ 成立。}$$

其余情形按**行锚**、**根**、**父**的键 (k, q, p) 选择末列最大组，记作 (k, c, r) 。这与比较使用的**父**、**行锚**、**根**优先级不同。置

$$\ell = x - c > 0, \quad \phi_b(i) = \begin{cases} i, & i < c, \\ i + b\ell, & i \geq c. \end{cases}$$

切点是控制父 c ，不是 $\max(k, c)$ 。输出有 $x + n\ell$ 列。取以下各组的并集，再作根向下闭包及规范化。

1. 对 $b = 0, \dots, n$, 复制旧前缀 $G \upharpoonright x$ 。第 j 列的组 (h, p, q) 移到第 $\phi_b(j)$ 列的 $(\phi_b(h), \phi_b(p), \phi_b(q))$ 。包括行锚在内, 四个坐标全部移动。
2. 对 $b = 0, \dots, n-1$, 在第 $x + b\ell$ 列建立接缝。对每个旧末列组 (h, p, q) :
 - 若 $h < k$, 加入 $(\phi_b(h), \phi_b(p), \phi_b(q))$;
 - 若 $h = k$, 加入 $(\phi_b(h), \phi_b(p), \min(\phi_b(q), \phi_b(r) - 1))$, 但最后一个坐标非负时才加入;
 - 若 $h > k$, 不加入。
3. 同一接缝加入所有 $(h, \phi_b(c), \phi_b(c))$, 其中 $0 \leq h < \phi_b(k)$ 。

切点 c 以前的不变前缀由各份复制共享, 重复加入不会增加关系。来源块 $[c, x)$ 共出现 $n+1$ 份, 其间有 n 个接缝。新复制块的首列与它前面的接缝是同一列, 其中的组需要合并。

移动最大根后取根闭包, 会填入移动映射跳过的根坐标。不能把规则改成只移动旧的单个根、却省去中间新出现的根。

单次调用只有有限循环与有限坐标运算, 不递归展开新产生的图。定义有限不意味着输出很小或实际运行很快。

4. 种子、标准域与结构性质

定义

$$A_0 = 0, \quad A_1 = [\emptyset], \quad A_{n+1} = A_n \curvearrowright [(n-1, n-1, n-1)] \quad (n \geq 1), \quad \text{Top}[n] = A_n.$$

令 $D(H)$ 包含 H 及从它经过有限次展开得到的所有图, 忽略 $0 \rightarrow 0$ 自环。有限标准域为

$$U = \bigcup_{n < \omega} D(A_n).$$

完整记号为 $U \cup \{\text{Top}\}$ 。构造器或解析器只检查结构合法性, 不保证输入属于 U 。

展开保持结构合法性。对每个有限图 G , $G[n]$ 是 $G[n+1]$ 的完整列前缀; 若旧末列非空, 则是真前缀。旧末列以前的全部列不变。每次非零展开都严格降低上述列序。反复取第零项可得到每个有限前缀。

链接纸面证明在 $S = KP_\omega + \text{“存在不可数序数”}$ 中, 给所有结构合法有限图构造非零展开严格下降的序数秩, 并证明列序在 U 上良序; 加入最大外顶端仍良序。这里 KP 包含**完整集合归纳**, 但不含幂集或选择公理。普通 Lean 入口与编码一段 S -推导是不同交付, 详见论文的验证说明。

列序定理的标准域限制不能删除。全部合法图中有无穷降链

$$[\emptyset]^{m+1}[(0, 0, 0)] > [\emptyset]^{m+2}[(0, 0, 0)] > \dots$$

这不是展开链。本文不声称 ARD 与 Y、RPD、LRD 或 Ω -LRD3 的序型比较, 也不声称公理上界最优。

5. 计数与显示

NER 文件提供无损列表、计数序列和完整弧线图。计数序列只是显示, 不是单射编码、可逆输入格式或比较键。

对某一列, 固定左侧前缀, 把该列看作末列, 作一次正指标展开后丢弃新追加的列。重复这个局部操作, 直至保留位置的列为空, 再将其删除。该列的计数是包含最后一次删除在内的局部操作次数。空列计数为一。保留位置只涉及第零接缝和来源切点列, 因此局部操作与选用哪个正指标无关。正指标展开改写非空末列时, 该位置的计数恰好减一。

局部有限计数可证明终止, 它不模拟整个反复 [1] 的轨道。 $A_1, \dots, A_6$ 的计数序列依次是 1, 2, 6, 23, 104, 537 的各个前缀。计数本身不能证明全局良序。

弧线图显示整张图, 而不是截取局部。精确计数使用 BigInt; 若计数或绘图超限, 界面报告超限并建议查看列表, 不静默换成近似计数或局部图。

6. 软件使用

在 NER 的自定义记号入口导入 ARD-arcs.ne-rewritten.js, 选择 ARD-legacy。当前注册 ID 为 ard-adjacency-v01; 默认显示列表, “等价表示” 菜单提供 计数序列、弧线图、邻接表 (文字) 和 邻接表 (图)。脚本的资源保护不改变有限数学规则。

独立 Python 定义需要 Python 3.10+, 仅依赖标准库:

```
from ard import AnchoredRows, TOP

a = AnchoredRows.seed(3)
assert str(a) == "[[(0,0,0)][(1,1,1)]"
assert TOP[3] == a
assert a[0] == AnchoredRows(a.columns[:-1])
assert a[1] < a
assert a[1].columns == a[2].columns[:len(a[1].columns)]
```

AnchoredRows() 是零, AnchoredRows.finite(n) 有 n 个空列, TOP 是 AnchoredRows(None)。构造器检查坐标并规范化各组。kind 为 zero、successor 或 limit; 有限非零式当且仅当未列为空时是后继。fs(n) 与 [n] 相同。自然数输入拒绝负数及布尔值。

Python 定义不含解析、计数、绘图、缓存或资源保护。整数任意精度, 但大展开仍可能耗尽时间或内存, 不宜用于无界暴力下降搜索。